FaceCoin: A Local Proof-of-Work Ledger for an In-Browser Racing Economy

Blake La Pierre

Working paper on the FaceRace reference client · 11 November 2026

Abstract. FaceCoin is the unit of account of FaceRace, a kart game that runs in the browser. The reference client calls the same integer unit a chip. This paper specifies the ledger that client implements: a chain of single-block SHA-256 headers, an account replay with at most one transaction in each block, a block subsidy, an implicit faucet, three material tokens, a constant-product pool, hire and poach payments, and stake purses. It also records the claims the mechanism does not support. There is no agreed peer set and no proof that a sporting result happened. Balances the local replay has never touched are defined to be forty units. The signed message names a spending address, but verification does not require that address to be the hash of the signing key. Header advertisements omit the transaction body, and the persisted chain is a suffix that the loader cannot reattach once the genesis block has fallen out of the window. FaceCoin, as implemented, is a local accounting device for a game, not a settlement network and not a claim on anything outside the client.

This note describes the FaceRace client on the `modern` branch at commit `e6fa6fc`. Identifiers in monospace are the names in that source. FaceCoin is not offered for sale. Nothing in this paper is a forecast of value.

1. Introduction

FaceRace puts a driver's face on a helmet and races that kart over a small set of courses. Between races the same page hires mechanics, fits wings and a rudder, seats humans in a purse, and shows a list of blocks. Those actions are meant to survive a refresh of the page, and some of them are meant to be checkable by another browser. The instrument used for that purpose is an integer balance. In the interface the balance is a pile of chips. This paper calls the unit FaceCoin, abbreviated FACE, and states the correspondence once: one FACE is one chip. The rest of the paper uses FACE so that the unit has a name independent of the label on the button.

The interesting object is not the name. It is the gap between what a player can reasonably think a "coin" does and what the transition function does. A player who reads only the pit copy will learn that mining earns chips, that a purse pays its winner, and that a signature authorizes a payment. Each of those sentences is true of some path through the user interface. None of them is a theorem about the ledger. The contribution of this paper is to write the ledger down carefully enough that those distinctions are checkable against the source, and to stop there. There is no measurement study, no proposed monetary rule, and no claim that a later client will close the gaps below.

Section 2 places the design next to the systems it borrows from. Section 3 fixes the model. Sections 4 and 5 specify work and the state transition. Section 6 treats the labor market, the material pools, and the purses. Section 7 collects the properties that fail. Section 8 concludes.

2. Related work

Hashcash uses a partial SHA-1 preimage as a cost of sending mail, so that the cost falls on the sender rather than on a central quota server [1]. Bitcoin uses a partial SHA-256 preimage, computed twice, as the cost of extending a chain of blocks, and it treats the heaviest such chain as the history of a currency [2]. The Bitcoin backbone analysis makes the further assumptions that honest parties share a peer set, that their combined hash rate exceeds the adversary's by a stated margin, and that messages eventually arrive [3]. FaceCoin borrows the

shape of a hash-linked header and the idea of a subsidy for the party who finds it. It does not borrow a peer set, a common difficulty window, or a proof that honest work prevails. Citing the backbone theorems as if they applied here would be a category error. They are cited so that the missing hypotheses are visible.

Account nonces, as in Ethereum's account model [4], are the right comparison for the replay rule in Section 5. A constant-product pool, as specified for Uniswap v2 [5], is the right comparison for the material exchange, with the important difference that FaceCoin rounds in the pool's favor by an integer division and does not charge a separate swap fee. The Sybil attack [6] is the right comparison for the faucet: an identity that costs nothing can be minted until the subsidy attached to new identities dominates any balance that had to be earned. Local custody of a long-term key in a browser origin is a storage problem, not a consensus problem; the Web Storage specification already treats that origin as the trust boundary [7].

3. Model

A *client* is one browser origin running the reference page. Its ledger state is whatever that page obtains by replaying the blocks it has stored. Two clients are not assumed to hold the same blocks. A client may announce the tip of its chain on a public tracker cell named `facerace/chain`, and it may receive announcements. The announcement studied here is a header encoding. It is not a proof that the sender's state matches the receiver's.

A *wallet* is a P-256 key pair generated by the Web Cryptography API. The private key is stored as a JSON Web Key in `localStorage`. The address is the first eight bytes of SHA-256, written as sixteen hexadecimal characters, of the uncompressed public point (the sixty-five-byte form beginning with `04`). Faces, display names, and kart colors are not inputs to this derivation. A face is not a key, and a key is not a face.

The objects of the ledger are:

- FACE balances, one integer per address, plus two distinguished addresses, `dev-pool` and `bonus-purse`, which do not receive the faucet described below.
- A nonce per address, starting at zero in the replay.
- Karts, each with an owner, a mechanic, wing and rudder flags, and counters for wins, purse wins, and races entered.
- Jobs: for each mechanic, a boss, a wage, and a skill counter.
- Listings of karts and of material lots.
- Bags of three material tokens: oil, battery, and engine.
- Three constant-product pools, one per material, each holding a FACE reserve and a token reserve.
- Purses, keyed by size and stake, each with an epoch, a list of seated addresses, and a start time.

Genesis is fixed. Its parent hash is sixty-four zero hex digits. Its timestamp is 1731283200, which is 11 November 2024 00:00:00 UTC. Its miner field is `6661636500000000`, the eight bytes of the ASCII word "face" followed by zeros. Its nonce is zero and its declared work threshold is sixteen leading zero bits. The genesis hash is the hash of that header, computed by the same function as every later block. Genesis pays no subsidy and carries no transaction.

Table 1. Constants of the reference client.

Constant	Value	Role
Unit	1 FACE = 1 chip, integer	All payments in the transition
Faucet	40 FACE	Balance of an address not yet written, other than the two pools
Subsidy	5 FACE	Credited to the miner of every block of height at least 1
Starter bag	6 oil, 3 battery, 1 engine	Bag of an address not yet written
Work bounds	12 to 26 leading zero bits	After the first child, which is forced to 20
Retarget	+1 bit if the gap is under 8 s; −1 if over 45 s	One step per block, using the miner’s timestamp
Pool seeds	oil 80/40, battery 90/18, engine 120/8	FACE reserve / token reserve
Hire prices	Ada 12, Bram 12, Nox 16	Also the kart’s speed, turn, and grip coefficients
Parts	Wings 8 FACE and 2 oil; rudder 6 FACE and 1 battery	Flags on a kart the signer owns
Purses	2 to 12 seats; stake 4 or 20	Pot equals seats times stake
Persistence	Last 400 blocks	Suffix written to local storage

4. Headers and work

A header is packed into a single sixty-four-byte SHA-256 message block, not into a Merkle tree and not into a double hash. Bytes 0–31 are the parent hash. Bytes 32–35 are the timestamp as a big-endian unsigned integer. Bytes 36–39 are the height. Bytes 40–47 are the miner address. Bytes 48–51 are the nonce. Bytes 52–55 are a transaction tag. Byte 56 is the SHA-256 padding bit, and bytes 60–63 store the bit length 448. The digest is one compression of that block. A header *meets work b* when the digest has at least b leading zero bits. The expected number of trials at threshold b is 2^b, if the hash is treated as a random oracle, which is the usual modeling assumption and not a theorem about this particular packing.

The tag is the first four bytes of SHA-256 of a canonical transaction string, read as a big-endian integer. A block with no transaction uses tag zero. A later block is admissible only if its height is one more than its parent’s, its timestamp is at least its parent’s, its tag matches the transaction it carries, and its threshold equals the retarget of the parent:

$$
\begin{aligned}
&b' = 20 \quad \text{if the parent is genesis;} \\
&b' = \min(26, b+1) \quad \text{if the claimed gap is under 8 seconds and } b < 26; \\
&b' = \max(12, b-1) \quad \text{if the claimed gap is over 45 seconds and } b > 12; \\
&b' = b \quad \text{otherwise.}
\end{aligned}
\tag{1}
$$

The gap is the difference of the two timestamps in the headers. Nothing in the validity rule compares a timestamp to the reader’s clock, and nothing takes a median of earlier timestamps. A party who can grind nonces can also choose the timestamp. Choosing a timestamp far beyond the parent lowers the next threshold, down to twelve bits, which is 4,096 expected trials. Choosing a timestamp only slightly beyond the parent raises the threshold, up to twenty-six bits. The private incentive of a party who wants cheap blocks is therefore to stamp a large gap. This is not a difficulty adjustment in the sense of Bitcoin’s 2,016-block window [2]. It is a one-step controller driven by an input the miner supplies.

Fork choice inside one client is the chain of greatest height among headers that linked. A header whose parent is unknown is kept in an orphan buffer of at most thirty headers and is linked later if the parent arrives and the tag still matches. There is no weight, no uncles, and no rule that prefers more work over more height. Because the threshold only changes by one bit at a time, height and work are close, but they are not the same once two forks have lived through different retarget paths.

5. State transition

State is not stored. It is the result of replaying the linked chain from an empty book. For every block of height at least one, the replay credits five FACE to the miner address before it applies the transaction. That credit does not depend on whether a transaction is present. The status callback in the miner reports a reward of five only when the block it just accepted had no transaction, and zero otherwise. The callback and the replay disagree. The replay is the balance the rest of the page reads back by walking the chain. The callback is a toast.

An address that the book has never written, other than the two pools, is defined to hold forty FACE. The two pools are defined to hold zero. The first successful debit or credit writes an explicit entry, so the faucet is received once, at first touch, and not again. The same pattern applies to materials: the first touch of a bag creates six oil, three battery, and one engine. Creating a fresh key is therefore enough to obtain forty FACE and a starter bag. The cost of a fresh key is the cost of generating a P-256 pair. There is no work requirement on the faucet. Section 7 returns to what that implies.

Each address has a nonce. A transaction is applied only when its nonce is exactly one greater than the nonce stored for its `from` field, which is zero when that field has never been seen. On success the stored nonce becomes the transaction’s nonce. Transactions are totally ordered by the blocks that carry them, and a block carries at most one. Pending transactions that fail against the new book are dropped. At most twenty pending transactions are retained.

The canonical body, which is the byte string that is signed and the byte string whose hash supplies the tag, is the concatenation, separated by vertical bars, of kind, from-address, nonce, mechanic field, kart identifier, seller, price, and ask-nonce. Unused fields travel as empty strings. The signature is ECDSA over P-256 with SHA-256, in the raw pairing of integers produced by the Web Cryptography API, under the public point attached to the transaction as the coordinates `x` and `y`. Verification checks that signature and, for a purchase that carries a seller’s ask, a second signature over a sell string. Verification does not hash the point and compare it to the from-address. Section 7 states the consequence without dressing it as a discovery of a trick.

6. Payments, materials, and purses

6.1 Hires, poaches, and parts

Three mechanics are hardcoded. Ada costs 12 FACE and is quoted as long gearing, with speed 1.16, turn 0.92, and grip 0.88 relative to the stock kart. Bram costs 12 and is quoted as sticky tires, with speed 0.94, turn 1.20, and grip 1.18. Nox costs 16 and is quoted as a light frame, with speed 1.08, turn 1.08, and grip 1.04. A hire transaction must name that exact price and a kart identifier derived from the mechanic, the from-address, and the nonce. The price is debited from the signer’s named address and passed through the fee rule below with no previous employer. The kart is minted to that address, without wings or a rudder. If the mechanic has no boss, or the boss is already the hirer, the job record is updated. If someone else holds the job, a hire is rejected unless the transaction’s ask-nonce equals one, in which case the client’s intended path is to poach first. The kart’s coeffi-

cients affect the physics of the race. They are not recomputed from anything on the chain beyond these three rows.

A poach must name a price at least the maximum of the current wage and the mechanic’s list price, plus four FACE. The whole price is debited and then split by the fee rule, which does pay the previous boss. The job’s boss and wage change. Skill does not reset. Wings cost 8 FACE and 2 oil. A rudder costs 6 FACE and 1 battery. Either flag sticks once set. The FACE portion of a part is split by the fee rule with no previous boss. Oil and battery are destroyed, not deposited in a pool.

The fee rule on an amount A, given a previous address P which may be empty, is

$$\begin{aligned} &\text{buyout} = \lfloor 0.25A \rfloor \text{ if } P \text{ is set, else } 0; \\ &\text{dev} = \lfloor 0.40A \rfloor; \\ &\text{bonus} = A - \text{buyout} - \text{dev}. \end{aligned} \tag{2}$$

The buyout is credited to P, the dev term to `dev-pool`, and the bonus term to `bonus-purse`. The forty percent is taken from the gross, not from what remains after the buyout. On a poach, a previous boss receives a quarter, the developer pool receives two fifths, and the bonus pool receives what is left, which is about thirty-five percent before integer rounding. On a hire or a part, where there is no previous boss, the developer pool receives two fifths and the bonus pool three fifths. None of these coins is destroyed. A hire moves FACE from a player address into the two pools. It is not a burn.

6.2 Listings and transfers

An owner may list a kart at a positive price, withdraw that listing, or buy a listed kart. A buy either matches a standing chain listing or carries the seller’s signature over the price. The full price moves from buyer to seller. There is no fee on a kart sale. Material lots are different: the seller’s bag is debited when the lot is listed, the buyer pays price times amount, one tenth of that payment is taken as a fee and then split by equation (2) with no previous boss, and the seller receives the rest. A withdrawn material lot returns to the seller’s bag. A send transaction moves either a positive FACE amount or a positive material amount to another address. Sends of FACE are not fee’d.

6.3 The material pools

Each material has a pool `(x, y)` of FACE reserve and token reserve, seeded as in Table 1. A buy of a material spends $\Delta x \geq 1$ FACE and returns

$$\Delta y = \left\lfloor \, y \cdot \Delta x \, / \, (x + \Delta x) \, \right\rfloor, \tag{3}$$

provided the result is at least one. A sell of $\Delta y \geq 1$ tokens returns the symmetric quantity of FACE. This is the constant-product trade of [5] with the floor taken so that the pool keeps the remainder. There is no fee parameter on top of that floor. Because the floor is taken at every trade, the product $x\,y$ tends to rise, and a round trip does not restore the starting reserves. Reserves are local to the replay. They are not reserves of a contract that another client is obliged to honor.

6.4 Purses and crowns

A purse key is a size from 2 to 12 and a stake class. The small stake is 4 FACE and the high stake is 20. The pot is the product of size and stake, so the smallest purse is 8 FACE and the largest is 240, before the bonus described next. A seat transaction must name the current epoch and the exact stake, and it is rejected if that address is already seated or if the purse has started. The stake is debited and the address is appended. When the

seated count reaches the size, the purse records the block time as its start. A claim must name the pot, must come from a seated address, and must be applied only after that start time. The claim pays the claimant the pot plus a gift

$$\text{gift} = \min(\text{ bonus-pool balance}, \lfloor 0.25 \cdot \text{pot} \rfloor), \tag{4}$$

debits the gift from the bonus pool, increments that kart's purse-win counter and its mechanic's skill, and opens the next epoch with an empty seat list.

Nothing in that predicate mentions a finish line, a lap count, a set of human drivers, or a transcript of the race. The client that built the transaction may perfectly well offer the claim button only to the driver who finished first, and it may refuse to seat a bot. Those are policies of the page. A block that carries a claim from any seated address, for the pot, is a valid block. A crown transaction is similar and cheaper: its price must be zero, the named kart must be owned by the from-address, and the effect is to increment the ordinary win counter and the mechanic's skill. A crown is not evidence that a race occurred. It is evidence that a valid signature of that shape was mined into a block.

7. Properties the ledger does not have

7.1 The signing key is not required to be the address

Let K be a P-256 public point and let addr(K) be the FaceCoin address derived in Section 3. The transition accepts a transaction when the attached point verifies the signature over the body. The body contains a from-address A. The check that is not performed is

$$A = \text{addr}(K). \tag{5}$$

A signature under any key therefore authorizes whatever account the body names, including an account whose key the signer does not know. Nonces do not repair this. The nonce is stored under A, and the signer chooses both A and the nonce. This is the opposite of the authentication property a reader expects from [2] and from [4], where the key that signs is the key that holds the coins. The omission should be read as a property of this client, not as an invitation. The paper states it because a description that said "transactions are signed by the spending wallet" would be false.

7.2 New identities are paid

Until an address is written into the balance map, every replay reports that it holds forty FACE, unless it is one of the two pools. An adversary who can generate keys can therefore introduce as many funded addresses as they care to generate, subject only to packing those first-touch transactions into blocks. This is Douceur's attack [6] with a subsidy attached. Work raises the cost of confirming those transactions. It does not raise the cost of the balance itself. Any accounting identity that treats FACE as scarce has to treat the faucet as an open mint, or has to restrict attention to addresses that were funded by subsidy and by transfers after an explicit genesis allocation. The client does the former.

The starter bag is a second open mint, in oil, battery, and engine. A new address can sell that bag into the pool of equation (3) and receive FACE out of the seeded reserves. The seeds are finite, so this path is bounded by the reserves rather than by the number of keys. It is still an allocation that does not come from the subsidy.

7.3 Sporting results are not inputs

The race simulates positions, laps, and checkpoints in a separate module. The ledger never reads that module. Seats, claims, and crowns are ordinary transactions. A purse can pay an address that did not finish, and it can pay an address that never drove, provided the address bought a seat and a later block carries its claim. Conversely, a driver who did finish has no ledger proof that they finished. Skill and win counters are functions of those transactions, so they inherit the same gap. They are useful as a display of what the local chain has applied. They are not a ranking of races.

7.4 Clients do not have a common history

The object announced to peers is a packed header: height, threshold, nonce, time, miner, hash, parent, and tag. The transaction body is not in that packing. A receiver who does not already have the body cannot reconstruct the book, and a tag of thirty-two bits does not identify a body uniquely. Local acceptance of a block does check the body against the parent book, but only in the client that has the body. Two browsers can advertise tips that share a hash prefix and still disagree about balances, because the hash covers the tag and not the body. Even a client alone does not keep a durable history past a few hundred blocks. The page writes the chronological chain and then keeps the last four hundred blocks. On the next load it tries to link that suffix. The parent of the oldest kept block is absent, so the suffix remains a list of orphans, and the tip stays genesis. Balances fall back to the faucet. A ledger that forgets its parent cannot replay, and this one, as stored, eventually forgets its parent.

7.5 Custody is the origin

The private key and the chain both live in the origin’s local storage, in exportable form. Any script that can run in that origin can sign and can replace the stored chain. There is no hardware boundary, no passphrase in the reference client, and no recovery document. A cleared site storage is a loss of the key. A copied site storage is a copy of the key. This is the ordinary threat model of [7], and it is the one that applies.

Addresses are sixty-four bits. A search of 2^{32} candidate keys is the birthday bound for a collision with some existing address, and a targeted search against one published address has cost 2^{64} if the hash is uniform. Sixty-four bits is short of a collision-resistant identifier. Combined with the missing check (5), collision resistance of the address is not the binding that protects a balance anyway. It is recorded because the derivation looks like an account identifier and is shorter than one.

8. Conclusion

FaceCoin, in the client this paper describes, is the integer chip of a browser game, earned on a local hash chain at five units a block, issued again whenever a new address is first touched, and spent on mechanics, parts, materials, and purses whose rules are completely explicit. The same explicitness shows the limits. The chain does not authenticate the spending key as the spending address. It does not know who won a race. It does not force two browsers to share a book. It does not remember more than four hundred blocks in a form it can reload. Those are not aesthetic complaints about an early draft. They are the specification.

The useful reading of the system is therefore a narrow one. Inside a single page session, the transition function is a deterministic replay: the same blocks produce the same karts, the same pools, and the same purses, and the fee split is the arithmetic in equation (2), not a promise about who deserves a wage. That is enough to drive the garage and the purse table. It is not enough to call the result a currency shared by the drivers whose faces are on the karts. The faces remain a picture. The keys remain a browser secret. The coin remains a replay.

References

1. A. Back, “Hashcash — a denial of service counter-measure,” technical note, 2002. Available: http://www.hashcash.org/papers/hashcash.pdf
2. S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. Available: https://bitcoin.org/bitcoin.pdf
3. J. Garay, A. Kiayias, and N. Leonardos, “The Bitcoin backbone protocol: Analysis and applications,” in *Advances in Cryptology — EUROCRYPT 2015*, LNCS 9057, Springer, pp. 281–310, 2015. doi: 10.1007/978-3-662-46803-6_10
4. V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform,” white paper, 2014. The account-nonce comparison in this note refers to that account model, not to a claim that FaceCoin executes its virtual machine.
5. H. Adams, N. Zinsmeister, and D. Robinson, “Uniswap v2 core,” technical whitepaper, March 2020. Available: https://uniswap.org/whitepaper.pdf
6. J. R. Douceur, “The Sybil attack,” in *Peer-to-Peer Systems*, IPTPS 2002, LNCS 2429, Springer, pp. 251–260, 2002. doi: 10.1007/3-540-45748-8_24
7. Web Hypertext Application Technology Working Group, “HTML Living Standard: Web storage,” section on local storage. The citation is for the origin trust boundary, which is the custody boundary of the reference client. Available: https://html.spec.whatwg.org/multipage/webstorage.html
8. National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS 186-5, February 2023. doi: 10.6028/NIST.FIPS.186-5. The client uses ECDSA on curve P-256 through the Web Cryptography API, not a FIPS module.
9. National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS 180-4, August 2015. doi: 10.6028/NIST.FIPS.180-4. The client uses one SHA-256 compression of a padded sixty-four-byte header.